

Minimal Iterative Harnesses for Learned Reasoning: When a Tiny Transformer Becomes a Procedure Executor

Paper 0.9 — Model + Harness Computational Boundary

Matched Paper 0.85 protocol — August 2026

Abstract

Fixed Transformers have bounded forward computational depth, while many symbolic tasks have instance-dependent execution depth. We study the smallest external harness that turns a tiny Transformer from a predictor into an iterative controller, but require a five-machine comparison before attributing gains to tools: one-token output (M0), free autoregression (M1), structured scratchpad (M2), local proto-tool (M3), and explicit iterative harness (M4). The primary instrument exactly reuses Paper 0.85’s shared-vocabulary, held-out-pair implication chains. Its local R0 gate passes three seeds, yet the model-only frontier is three for one-token, free-trace, and fully supervised structured generation. Completed learned M3/M4 controllers at L2/W64/H4 also have final-answer frontier three and zero acquisition probability at unseen $K = 4$. Exact nested continuations to 2,000 and 4,000 updates do not move this boundary: M4 remains exactly zero at every tested $K \geq 4$ for all seeds, while M3 stays near chance. Thus neither the local tool, explicit loop, nor fourfold optimization budget expands the measured frontier in this baseline architecture. Oracle T1/T2/T3 executions remain infrastructure controls, not learned evidence, and the staged study proceeds to allocation controls.

1 Core idea

A fixed Transformer computes $y = F_\theta(x)$. The minimal harness permits

$$s_{t+1} = F_\theta(x, s_t),$$

where output may contain a special action token:

$$\langle CALL, op, args \rangle.$$

The harness deterministically evaluates the primitive, appends its typed result, and invokes the same model again.

The effective machine transition must be reported explicitly. M0 performs one final-token prediction. M1 emits an unconstrained proposition trace. M2 emits typed STATE/DERIVE/ANSWER tokens but receives no semantic callback. M3 may issue one local tool call; M4 repeatedly selects calls or an answer under an explicit step budget. A comparison that skips M1/M2 cannot distinguish external semantic execution from writable-state or supervision effects.

2 Matched propositional prerequisite and dataset

The primary experiment reuses Paper 0.85’s corrected R0-v2 and recurrence Stage1-v2. Its 64-symbol vocabulary is shared across train and test and every symbol has positive input/output training coverage. Ordered implication pairs, rather than lexical items, are disjoint: 3,226 train pairs and 806 held-out pairs with zero observed leakage. The three L2/W64/H4 R0 seeds pass entailed, consequent-swap, fact-mismatch, and rule-LHS controls at 98.8–100%. An earlier all-new-symbol condition is only a lexical/OOV-output stress and is not the local-rule prerequisite.

Proof training is bounded to $K \leq 3$. The matched initial test uses $K \in \{1, 2, 3, 4, 6, 8\}$ with held-out edges and a query instruction that does not contain the target. An earlier query-target pilot is quarantined as invalid. Future shuffled-rule, distractor, branching, and ICL cells must preserve this generator rather than substitute an easier logic task.

3 Required machine ladder and current evidence

For matched architecture and task cells define

$$K_{\text{tok}}, K_{\text{AR}}, K_{\text{scratch}}, K_{\text{tool}}, K_{\text{harness}}.$$

Table 1 distinguishes observed learned-model evidence from semantic infrastructure controls.

Machine	Current condition	Measured frontier	Status
M0	O0 final-only	3	observed learned model
M1	O1 free trace	3	observed learned model
M2	O2 structure, no state labels	none	observed learned model
M2	O3 full derivation labels	3	observed learned model
M3	one selected local T1 call	3	observed learned model
M4	iterative T1 controller+harness	3	observed learned model

Table 1: Evidence status for the matched ladder. M3/M4 use three learned seeds; oracle-controller validation is reported separately.

M0, M1, and fully supervised M2 are perfect at $K = 1, 2, 3$ for every seed, then fail the three-seed gate at $K = 4$; the $K = 4$ acquisition fraction is zero for every condition. Hence

$$K_{\text{tok}} = K_{\text{AR}} = K_{\text{scratch}, \text{O3}} = 3, \quad G_{\text{recurrence}} = 0.$$

O1 transition accuracy falls to 23–32% at $K = 4$ and approximately 10–20% at $K = 6, 8$, with degraded termination. O2 often learns formatting/termination but no exact trajectories. This supplies a sharp model-only boundary against which the learned M3/M4 results below are compared.

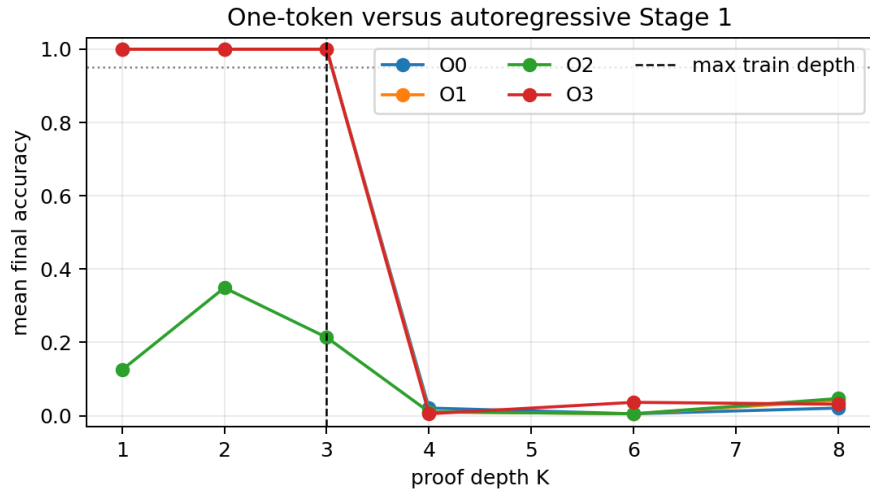


Figure 1: Observed Paper 0.85 model-only baselines reused unchanged. The learned M3/M4 comparison uses the same split, architecture, seeds, and depth axis.

4 Minimal proto-tool interface

Use the smallest action grammar:

$$\langle \text{CALL}, \text{op}, \text{arg}_1, \dots, \text{arg}_n \rangle$$

and terminal

$$\langle \text{ANSWER}, y \rangle.$$

No planner, search algorithm, hidden chain-of-thought, retrieval system, or extra LLM is allowed in the primary tiny-model experiment.

Tool strength is a controlled resource. T1 evaluates only a controller-selected pair, $\text{apply}(\text{current}, \text{rule}) \rightarrow \text{next}$, and performs no search. T2 returns rules applicable to the current state, changing the selection/search burden. T3 returns the certified next proof state and is an upper control that moves most reasoning into the tool. The central condition begins with T1; T2 is introduced only to diagnose search, and T3 can never support the central model claim.

5 CPU-validated harness semantics

The corrected comparison infrastructure generates 384 Stage1-v2 test chains (64 at each of six depths). An explicitly oracle selecting controller executes each under T1, T2, and T3, producing 1,152 exact reference runs. All terminate correctly; calls equal K and model-invocation placeholders equal $K + 1$ by construction. These facts validate typed transitions, termination, and cost logging only. They do not estimate learned rule selection, call arguments, result integration, or stopping.

These oracle rows are quarantined from the learned comparison. They establish that a correct policy can use each API and that cost fields have the intended meaning, but do not fill any learned M3/M4 cell.

6 Learned T1 controllers

The first learned comparison fixes L2/W64/H4, 1,000 updates, seeds 11/23/37, training depths $K \leq 3$, and the six held-out-pair test depths. M3 receives exactly one opportunity to select a local edge; the deterministic T1 primitive returns the selected next proposition or an invalid token, after which the model emits a final answer. M4 repeatedly emits either the next proposition for T1 or ANSWER; the harness performs no search. Every condition has 64 examples per depth and seed, for 2,304 preserved prediction rows.

6.1 Metric correction

The original M3 export named

$$\frac{\mathbf{1}\{\text{selected edge valid}\}}{K}$$

“transition accuracy.” This is not an accuracy over attempted transitions because M3 attempts only one call. We retain the predictions and relabel the quantity *one-call edge coverage*. M3 is reported with selected-edge validity, one-call coverage, post-tool final-answer accuracy, and invalid-call rate. M4 genuinely attempts a trajectory and retains per-transition accuracy, exact-trajectory/final accuracy, termination precision and recall, invalid calls, and nontermination. M3 has no learned recurrent stopping decision, so assigning it a termination score would also be misleading.

Machine	K	Final	Execution metric	Worst seed final	P_{acquire}	Gate
M3	1	1.000	1.000	1.000	1.00	pass
M3	2	1.000	.500	1.000	1.00	pass
M3	3	1.000	.333	1.000	1.00	pass
M3	4	.021	.076	.000	.00	fail
M3	6	.005	.045	.000	.00	fail
M3	8	.016	.049	.000	.00	fail
M4	1	1.000	1.000	1.000	1.00	pass
M4	2	.995	.997	.984	1.00	pass
M4	3	1.000	1.000	1.000	1.00	pass
M4	4	.000	.000	.000	.00	fail
M4	6	.000	.012	.000	.00	fail
M4	8	.000	.054	.000	.00	fail

Table 2: Completed learned T1 results. M3’s execution metric is one-call edge coverage; M4’s is per-transition accuracy. Acquisition requires final accuracy and the machine-specific execution gate to reach .95 in a seed.

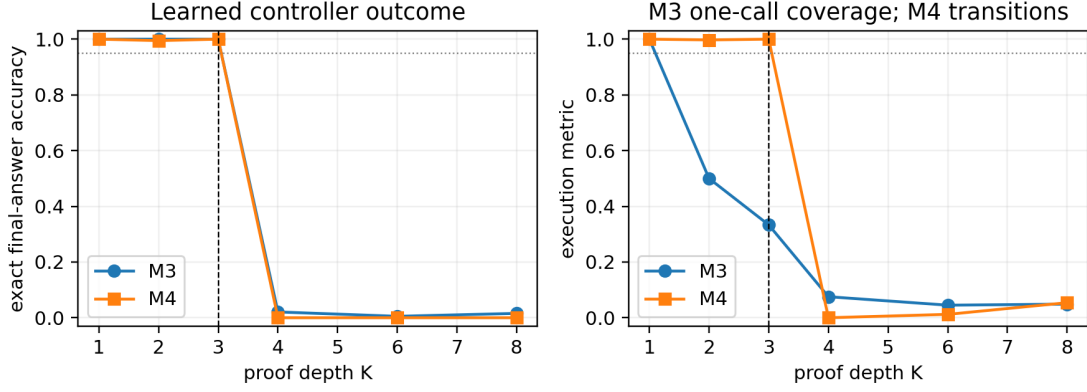


Figure 2: Learned T1 controllers. The dashed vertical line is the training boundary. M3’s declining coverage inside the passing range is structural—one call covers $1/K$ of a valid chain—while M4’s curve measures all required transitions.

Both machines therefore have the contiguous frontier

$$K_{\text{tool}} = K_{\text{harness}} = 3, \quad G_{\text{tool}} = G_{\text{harness}} = 0$$

relative to the matched M0 frontier. M3 selects a valid first edge on every training-range evaluation. Its perfect $K = 2, 3$ answers cannot be attributed to tool execution of the complete proof: the one call covers only one half or one third of the chain, and the model predicts the remainder. At unseen depths, selected-edge validity itself becomes unstable (mean .302, .271, and .391 at $K = 4, 6, 8$), while exact answers fall below 2.1%.

M4 provides the stronger semantic machine but does not acquire a longer procedure. It is exact at $K = 1, 3$ and reaches 99.5% mean final accuracy at $K = 2$ (worst seed 98.4%). At every tested $K \geq 4$, exact final and trajectory accuracy are zero. Invalid-action rates are 100% at $K = 4, 6$ and 98.96% at $K = 8$ when pooled across seeds; no example is classified as a max-step nontermination. Correct termination recall is zero beyond the training boundary because invalid actions usually end the trajectory before a valid stop. This is a train-short/test-long acquisition failure, not evidence that T1 or an ideal iterative controller lacks the representational ability to traverse longer chains.

7 Later extension: tree primitives

Begin with one primitive:

$$\text{parent}(x) \rightarrow y$$

plus a root/no-parent signal. Add children, equality, or level primitives only if necessary.

8 Later relational tasks

Compare model-only with model+harness on:

$$\begin{aligned} &\text{parent}(x), \text{grandparent}(x), \text{root}(x), \text{isAncestor}(x, y), \\ &\text{sameParent}(x, y), \text{sameGrandparent}(x, y), \text{sameAncestorAtLevel}_k(x, y). \end{aligned}$$

Vary tree depth, branching, distractors, and required traversal independently. These experiments follow the propositional M0–M4 comparison. Existing deterministic tree reference controllers validate the harness API but are not learned-model evidence.

9 Closure hypothesis

For a fixed model, $d_{\text{max}} < \infty$ may persist even with scale. For a harnessed model, if local transition competence is reliable,

$$T_{\text{steps}} \approx O(d)$$

should permit much larger relation depth. The key question is whether the learned controller generalizes:

call parent; update state; test stop; repeat.

10 Cross-paper benchmark families

Paper 0.5 supplies bounded pattern/refinement tasks; Paper 0.6 supplies semantic/tree relations; Paper 0.7 supplies propositional proof chains and first-order unification; Paper 0.8 supplies copy, contextual task selection, and structured rule induction. Not every task requires a tool: the objective is to determine where iteration changes the competence class.

11 Tiny-model phase diagram and staged next experiment

Vary model resources L, W, H and harness budget $T_{\max}$. Compare: model-only, one callback, bounded loop, and loop until a learned stop condition.

The completed three-seed result licenses a bounded resource diagnostic, not the old 1,620-cell factorial. The v3 staged plan first tests exact $2\times/4\times$ continuation of L2/W64/H4, then depth and parameter-matched deep/narrow versus shallow/wide allocation. Width/head controls are conditional; training-example diversity at fixed $K_{\max} = 3$ and a separate $K_{\max} = 3$ versus 4 boundary test follow only for the selected architecture. Each stage stops on rescue or futility gates.

Stage A replays from step zero under the v3 exact checkpoint contract and takes nested snapshots at 2,000 and 4,000 updates. Checkpoints preserve model and optimizer state, Python/NumPy/Torch/MPS RNG state, the data-stream cursor, and configuration/dataset hashes; an interrupted-versus-uninterrupted CPU test produces bitwise-identical weights and identical metric rows.

Machine	Updates	$K \leq 3$ final	$K = 4$ mean/worst	Final frontier
M3	2,000	1.000	.016/.000	3
M3	4,000	1.000	.010/.000	3
M4	2,000	1.000	.000/.000	3
M4	4,000	1.000	.000/.000	3

Table 3: Completed three-seed Stage-A budget continuation. The frontier shown is final-answer competence. Under the stricter machine-specific execution gate, M3’s one-call coverage is structurally below .95 for $K > 1$, while M4’s frontier remains three.

More optimization preserves acquisition on the training support but does not produce procedure-length extrapolation. M3’s mean final accuracy at $K = 4$ changes from .016 at 2,000 updates to .010 at 4,000; M4 is zero at $K = 4, 6, 8$ under both snapshots for every seed. Stage A satisfies the preregistered completion prerequisite for Stage B, but not a rescue claim. The next bounded test changes model depth/allocation rather than spending further budget on this baseline.

12 Minimality

Shrink model size, special-token grammar, tool primitives, state fields, and maximum steps. The target is the smallest model+harness pair that exhibits systematic depth extrapolation unavailable to the model alone.

13 Reproducibility and evidence boundary

Matched generator construction is in `recurrence_chains.py`. Typed T1/T2/T3 semantics are in `proof_harness.py`; the comparison and status runners use the `paper09_paper085_*` prefix. Configuration `paper085_comparison_v2.json` pins the corrected shared-vocabulary split. Artifacts under `results/paper085_comparison_v2` contain the oracle-controller validation and explicitly record `learned_model_results=false`.

Learned checkpoints and immutable legacy exports are under `results/learned_controller_v1`. The semantic-v2 analysis preserves all 2,304 predictions, pins the source SHA-256, and writes corrected raw rows, seed/depth summaries, three-seed frontiers, a manifest, and Figure 2 under `analysis_v2`. Configurations `learned_controller_v3_`

`staged.json` and `learned_controller_v3_cells.csv`, with the companion plan manifest, preregister the bounded stages. The Stage-A runner is `paper09_learned_controller_stage_a.py`; tracked artifacts under `results/learned_controller_v3/stage_a_budget` contain 4,608 prediction rows, per-seed/depth summaries, losses, and an explicit completion manifest. Checkpoint tensors are excluded from version control.

14 Appendix: pretrained models

Use small pretrained models only as external validation, separating direct answer, proto-tool harness, native tool calling, and native reasoning/scratchpad modes where available. Corpus exposure and learned procedures remain uncontrolled.

15 Conclusion

Paper 0.9 asks where systematicity comes from when the same learned local operator is embedded in a minimal iterative machine. Local implication is reliably acquired, but one-token, free-trace, fully supervised structured, one-call T1, and iterative T1 machines all stop at measured depth three in the initial matched regime. Exact continuation through four times the original optimization budget leaves that boundary unchanged. The learned M4 harness changes the available computation but does not yield acquisition of a train-short/test-long controller in the baseline allocation; M3 shows why tool calls must be decomposed, since a valid one-edge call can coexist with neural completion of the rest of a bounded proof. These are finite negative acquisition results, not impossibility or closure claims. The next question is whether architecture allocation, diversity, or the training-depth boundary moves the frontier under the remaining preregistered gates.